PONTIFÍCIA UNIVERSIDADE CATÓLICA DO PARANÁ
ESCOLA POLITÉCNICA
BACHARELADO EM ENGENHARIA DE COMPUTAÇÃO

ANDRÉ FABRICIO WOZNIACK
RICARDO HESPANHOL PAMPLONA

REMEDIAÇÃO CONTEXTUAL DE FALHAS DE AUTORIZAÇÃO EM APIS ASP.NET CORE COM LLM E RAG

PROJETO FÍSICO – 2º BIMESTRE

Orientador: Prof. Julio Cesar Nievola

Visto do Orientador

CURITIBA
2026

RESUMO

A correção de falhas de autorização em APIs depende das regras de acesso da aplicação. Um alerta de análise estática pode indicar o endpoint afetado, enquanto a política aplicável, seus requisitos e o mecanismo de autorização por recurso podem estar definidos em outros arquivos. Este projeto propõe uma ferramenta de linha de comando em C# para sugerir correções de autorização ausente ou incorreta em APIs ASP.NET Core. A ferramenta recebe um alerta em SARIF ou um manifesto experimental, analisa o código com Roslyn e recupera definições de políticas, *handlers* e implementações equivalentes do próprio repositório. Um modelo de linguagem gera uma proposta em formato *diff*, acompanhada de justificativa e referências ao código. A proposta é avaliada em ambiente isolado por compilação, testes funcionais, testes de autorização e revisão de aderência às regras do projeto. O experimento compara a geração com o método afetado, com o arquivo completo e com o arquivo acrescido de contexto recuperado. A métrica principal é a proporção de propostas simultaneamente seguras, funcionais e aderentes ao modelo de autorização. A avaliação começa com um caso de autorização por recurso e prossegue com um piloto de cinco a dez casos.

Palavras-chave Autorização. ASP.NET Core. Remediação de vulnerabilidades. Modelos de linguagem. Geração aumentada por recuperação.

SUMÁRIO

1 INTRODUÇÃO 3
1.1 CONTEXTO E PROBLEMA 3
1.2 TRABALHOS RELACIONADOS 3
1.3 PERGUNTA DE PESQUISA E HIPÓTESE 4
1.4 OBJETIVOS 4
1.4.1 OBJETIVO GERAL 4
1.4.2 OBJETIVOS ESPECÍFICOS 4
1.5 ESCOPO E ESTRUTURA DO DOCUMENTO 5
2 DETALHAMENTO DO PROJETO 6
2.1 VISÃO GERAL 6
2.2 INTERFACES ENTRE OS BLOCOS 6
2.3 ANÁLISE DO CÓDIGO 7
2.4 RECUPERAÇÃO CONTEXTUAL 8
2.5 GERAÇÃO DA PROPOSTA 8
2.6 VALIDAÇÃO E RELATÓRIO 9
2.7 CONJUNTO DE CASOS E PILOTO 9
3 CRONOGRAMA DO PROJETO 10
4 PROCEDIMENTOS DE TESTE E VALIDAÇÃO DO PROJETO 11
4.1 TESTES DE CAIXA PRETA 11
4.2 TESTES DE CAIXA BRANCA 11
4.3 CRITÉRIOS DE AVALIAÇÃO 12
4.4 DESENHO EXPERIMENTAL E MÉTRICAS 12
5 ANÁLISE DE RISCOS 14
6 CONCLUSÃO 15
REFERÊNCIAS 16

1 INTRODUÇÃO

1.1 CONTEXTO E PROBLEMA

Ferramentas de análise estática de segurança (*Static Application Security Testing*, SAST) produzem alertas que ajudam o desenvolvedor a localizar código que merece revisão. No caso da autorização, determinar a correção exige conhecer quais identidades podem realizar uma operação e em quais recursos. Essas regras podem estar distribuídas entre a configuração da aplicação, os metadados do endpoint e os serviços que decidem o acesso. Uma sugestão baseada apenas no alerta pode restringir uma operação a um papel inadequado ou permitir que uma identidade autenticada acesse um recurso para o qual ela não tem permissão.

O ASP.NET Core permite declarar políticas em controladores, ações e endpoints, além de avaliar a autorização explicitamente por meio de `IAuthorizationService`. Políticas podem combinar papéis, *claims* e requisitos avaliados por *handlers*. Quando a decisão depende da entidade carregada, como um documento associado a um proprietário, a verificação precisa levar esse recurso em conta (Microsoft; Microsoft, s.d.c, s.d.d). A correção precisa, portanto, considerar a operação, a política e as convenções do repositório.

O recorte adota a CWE-285 como categoria de referência e utiliza CWE-862 e CWE-863 para classificar os casos. A CWE-862 descreve a ausência da verificação exigida. A CWE-863 descreve uma verificação existente que decide o acesso de modo incorreto. A autorização por recurso pode aparecer em ambas as classes, conforme a natureza da falha observada (MITRE; MITRE; MITRE, s.d.a, s.d.b, s.d.c).

1.2 TRABALHOS RELACIONADOS

A geração aumentada por recuperação (*Retrieval-Augmented Generation*, RAG) acrescenta informações recuperadas à entrada do modelo de linguagem (Lewis *et al.*, 2020). Neste trabalho, essas informações são trechos de código e relações entre símbolos do repositório avaliado. O PatchAgent integra localização, geração e validação de correções (Yu *et al.*, 2025). O GitHub também oferece sugestões de correção em seu fluxo de alertas de análise de código (GitHub, 2026). Esses trabalhos

estabelecem precedentes para a remediação assistida. A questão deste projeto é medir o efeito da recuperação orientada ao modelo de autorização do ASP.NET Core, mantendo fixos o modelo de linguagem e o procedimento de avaliação.

1.3 PERGUNTA DE PESQUISA E HIPÓTESE

> O contexto recuperado de políticas, requisitos, *handlers* e implementações equivalentes melhora a proporção de correções seguras, funcionais e aderentes ao modelo de autorização de APIs ASP.NET Core, em comparação com a geração sem recuperação?

A hipótese é que acrescentar contexto de autorização ao arquivo completo aumenta a proporção de correções que atendem simultaneamente aos três critérios, sem reduzir a proporção de correções seguras e funcionais. O contraste principal será entre o arquivo completo e o arquivo acrescido de contexto recuperado. A condição com apenas o método afetado permitirá observar quanto do ganho pode ser atribuído à leitura do arquivo.

1.4 OBJETIVOS

1.4.1 OBJETIVO GERAL

Desenvolver e avaliar uma ferramenta de linha de comando que sugere correções de autorização em APIs ASP.NET Core com apoio de um modelo de linguagem e do contexto recuperado do repositório.

1.4.2 OBJETIVOS ESPECÍFICOS

a) Normalizar alertas SARIF e manifestos experimentais e localizar os endpoints correspondentes com Roslyn.

b) Recuperar as políticas, os requisitos, os *handlers*, os serviços e as implementações relevantes para a operação afetada.

c) Gerar propostas de correção em formato *diff*, com justificativa, referências e indicação de incerteza.

d) Validar as propostas em cópias isoladas e registrar evidências separadas de segurança, funcionalidade e aderência arquitetural.

e) Comparar as condições de geração em um piloto reproduzível e avaliar a viabilidade de ampliar o conjunto de casos.

1.5 ESCOPO E ESTRUTURA DO DOCUMENTO

O protótipo abrange APIs ASP.NET Core em C#, com controladores e Minimal APIs, e trata autorização ausente, política incorreta e falhas de autorização por recurso. A ferramenta atua na remediação após a indicação de um caso por um SAST ou pelo manifesto experimental. O escopo exclui a criação de um detector genérico, o suporte a outras linguagens ou classes de vulnerabilidade, a integração com IDE e a aplicação automática no repositório original.

O Capítulo 2 descreve a arquitetura e os casos. O Capítulo 3 apresenta o cronograma. O Capítulo 4 define os testes e o experimento. O Capítulo 5 analisa os riscos, e o Capítulo 6 discute a viabilidade do recorte.

2 DETALHAMENTO DO PROJETO

2.1 VISÃO GERAL

A ferramenta será uma aplicação de linha de comando em C# para .NET 10. A Figura 1 mostra o fluxo para um caso. Cada proposta parte de um commit identificado. A recuperação é usada na condição C do experimento, enquanto as condições A e B passam diretamente da análise à geração. A validação e a emissão do relatório são comuns às três condições.

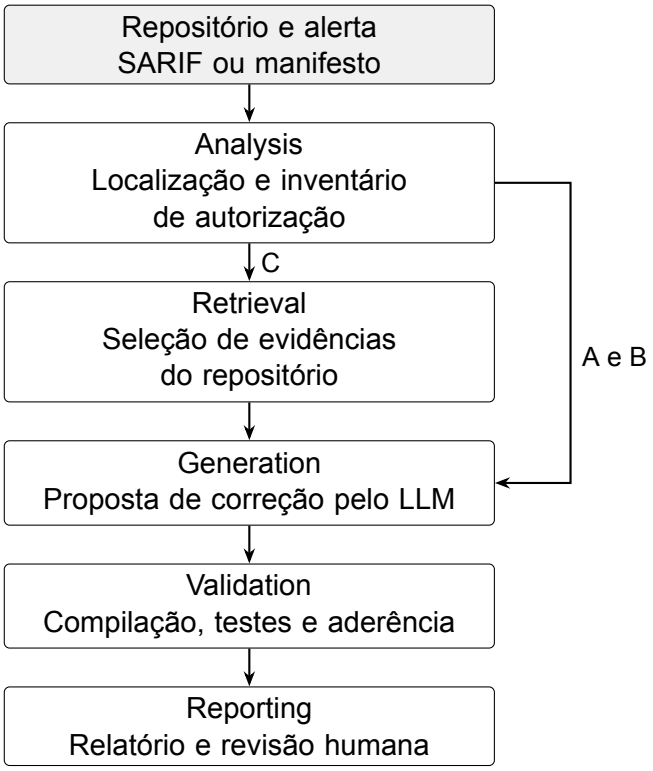


Figura 1 – Fluxo do protótipo de remediação.

Fonte elaborada pelos autores.

2.2 INTERFACES ENTRE OS BLOCOS

Analysis Recebe o repositório, o commit e o SARIF ou manifesto. Produz um alerta normalizado, a localização do endpoint e um inventário de autorização.

Retrieval Recebe o endpoint e o inventário. Retorna trechos com arquivo, intervalo de linhas, símbolo e justificativa de relevância.

Generation Recebe o alerta e o código permitido pela condição, incluindo as evidências recuperadas quando aplicável. Retorna *diff*, justificativa, referências, incerteza e dados de uso do modelo.

Validation Recebe a proposta, o commit e a configuração de avaliação. Retorna o estado da aplicação do *diff*, da compilação e dos testes, além dos itens para revisão arquitetural.

Reporting Reúne as entradas, a proposta e os resultados de validação. Emite o registro da execução e o material para revisão humana.

2.3 ANÁLISE DO CÓDIGO

A entrada principal será SARIF 2.1.0 (GitHub, s.d.). O importador extrairá identificador da regra, mensagem, arquivo e região. Uma tabela de correspondência, configurada para cada fonte, associará regras conhecidas a CWE-862 ou CWE-863. Quando a cobertura do analisador for insuficiente, um manifesto experimental indicará identificador do caso, commit, CWE, arquivo e intervalo do endpoint. A origem da entrada permanecerá registrada no relatório. Localização ambígua, classe desconhecida ou caminho externo ao repositório interrompem a geração e produzem um diagnóstico.

Em ambiente isolado, com SDK e dependências preparados, o Roslyn carregará o projeto, localizará a região informada e consultará árvores sintáticas e modelos semânticos (Microsoft, s.d.b). Para controladores, serão registrados classe, ação, rota e método HTTP. Para Minimal APIs, serão registrados o mapeamento, o grupo de rotas, quando resolvível, e a função ou expressão que implementa o endpoint.

O inventário reunirá os metadados de `Authorize`, `AllowAnonymous` e `RequireAuthorization`, os registros feitos por `AddAuthorization`, as políticas padrão e de *fallback*, papéis, *claims*, implementações de `IAuthorizationRequirement` e `IAuthorizationHandler` e chamadas a `AuthorizeAsync`. Os vínculos serão associados ao nível em que atuam, como configuração global, grupo, controlador, ação ou serviço. Registros condicionais, políticas dinâmicas e dependências que não puderem ser resolvidas serão sinalizados para revisão. A presença de um atributo isolado é insuficiente para classificar a autorização efetiva de um endpoint.

2.4 RECUPERAÇÃO CONTEXTUAL

O recuperador usa o inventário para partir do endpoint e seguir referências a símbolos que definem suas políticas, seus requisitos, seus *handlers* e os serviços de autorização. Uma busca textual complementa os vínculos estruturais para localizar endpoints que operam sobre o mesmo tipo de recurso ou aplicam verificações comparáveis. Os candidatos são ordenados por ligação semântica direta, equivalência da operação e, por último, correspondência textual. O bloco elimina duplicatas e respeita um orçamento de contexto fixado no piloto. Para cada trecho, registra a origem e o motivo da seleção.

Um padrão usado em outro endpoint será considerado evidência somente quando a operação e as condições de acesso forem compatíveis. Por exemplo, editar um documento pode exigir um requisito associado à entidade carregada, enquanto listar documentos pode depender de outra regra. Trechos contraditórios ou insuficientes serão identificados como tal. O primeiro protótipo usará recuperação estrutural e textual no repositório local.

2.5 GERAÇÃO DA PROPOSTA

A condição experimental determina se a entrada contém o método ou manipulador associado, o arquivo completo ou o arquivo completo acrescido dos trechos recuperados. As três condições compartilham o mesmo modelo, os parâmetros de geração, as instruções e o formato de saída. O adaptador de LLM será selecionado por configuração. Antes do envio, a CLI limita o conteúdo aos arquivos autorizados e verifica a presença de segredos e configurações sensíveis. Casos que exigirem conteúdo indisponível serão sinalizados.

O modelo deve produzir um *diff* unificado, explicar qual regra foi aplicada e apontar as evidências disponíveis. Quando não houver evidência suficiente para sustentar uma política ou condição de negócio, a saída prevista é uma indicação de incerteza, com possibilidade de abstenção. Os testes do avaliador e a correção de referência ficam fora da entrada. A saída é interpretada como dados e passa por validação de formato antes de qualquer tentativa de aplicação.

2.6 VALIDAÇÃO E RELATÓRIO

A validação cria um Git *worktree* temporário no commit do caso. O *diff* deve ser aplicável e limitado aos arquivos de produção autorizados. Testes e configurações ficam protegidos. Em contêiner sem rede, com dependências preparadas, recursos limitados e sem credenciais, o validador executa `dotnet build` e os testes funcionais e de autorização. Registra o resultado, a duração e o motivo de falha de cada etapa e descarta a cópia de trabalho.

A revisão de aderência confronta o *diff*, as evidências e a regra esperada, conforme a Seção 4.3. O relatório JSON registra alerta, commit, condição, localização, contexto, proposta, estado de validação, tempos e dados do modelo, incluindo custo quando disponível. A versão em Markdown apoia a revisão humana. A decisão de aplicar a correção cabe ao desenvolvedor.

2.7 CONJUNTO DE CASOS E PILOTO

Cada caso registra origem, commit, endpoint, CWE, evidências relevantes, regra de acesso, testes de segurança e funcionais e correção de referência. O manifesto de localização fica separado do material de avaliação. A correção de referência documenta uma solução válida, e os testes admitem outros *diffs* que satisfaçam os critérios.

O primeiro caso usará uma API de teste para documentos. Uma operação de alteração depende da regra de propriedade já aplicada em um endpoint equivalente por `IAuthorizationService` e um *handler*. A avaliação verifica se a proposta reutiliza esse mecanismo, preserva o acesso autorizado e impede efeitos indevidos quando o acesso é negado.

O piloto reunirá cinco a dez casos em duas a três aplicações públicas ou de teste, cobrindo ausência, política incorreta e autorização por recurso. A ampliação para 30 a 50 casos depende de projetos reproduzíveis, regras de acesso documentadas e entradas SARIF ou manifestos.

3 CRONOGRAMA DO PROJETO

As três primeiras etapas abrangeram o formulário e a proposta, a entrega do Plano de Projeto em 25/09/2026 e a defesa em 28/09/2026. O cronograma da Figura 2 mantém os marcos acadêmicos e reorganiza o desenvolvimento para começar pela análise de autorização e por um caso piloto executável. As barras verdes indicam atividades concluídas no cronograma anterior. As barras azuis indicam atividades planejadas nesta revisão.

A validação será preparada junto ao primeiro caso, antes das execuções sem RAG. Em seguida, serão implementadas a recuperação contextual e a comparação das três condições. A ampliação após o piloto dependerá da reprodução dos casos, do custo das execuções e da disponibilidade de evidências para avaliar a aderência.

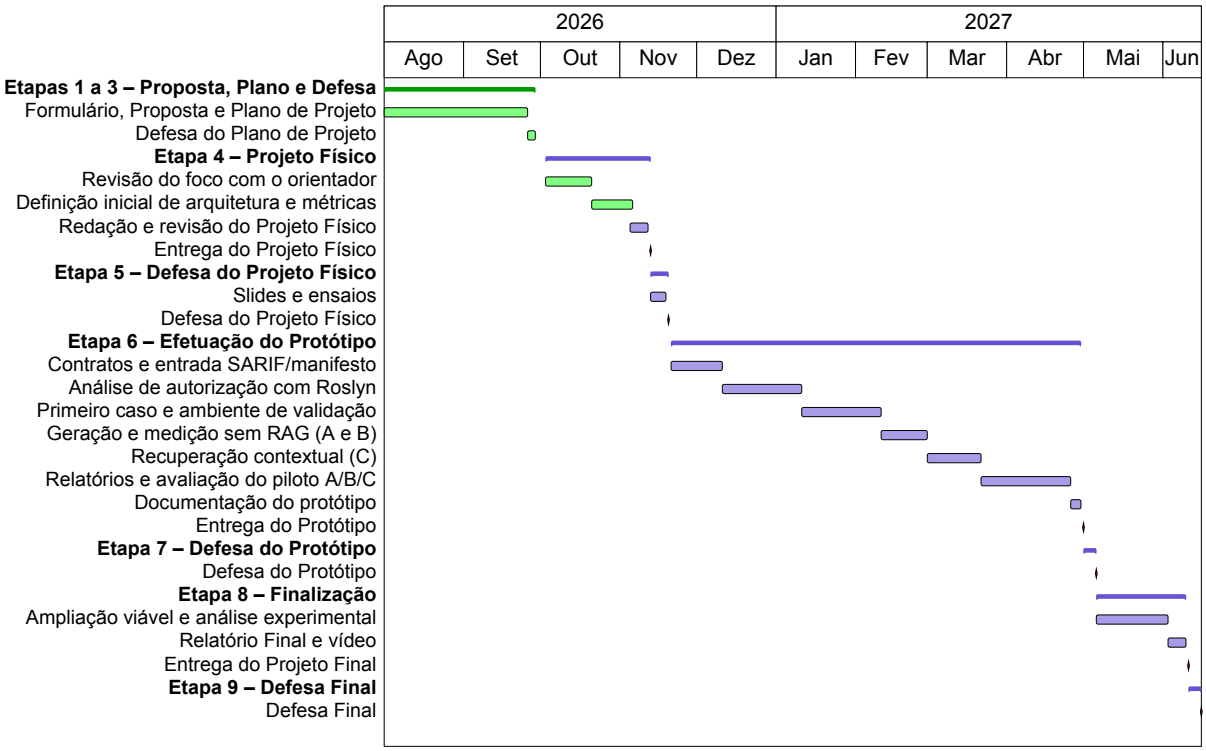


Figura 2 – Cronograma revisado do projeto – Gráfico de Gantt.

Fonte elaborada pelos autores.

4 PROCEDIMENTOS DE TESTE E VALIDAÇÃO DO PROJETO

4.1 TESTES DE CAIXA PRETA

Os testes exercitam a CLI pela interface disponível ao desenvolvedor. Para os cenários de falha controlada, será usado um provedor substituto de LLM, com respostas previamente definidas. Os resultados desses testes verificam o protótipo. As métricas experimentais serão calculadas sobre execuções com o modelo escolhido para o estudo.

Tabela 1 – Cenários de caixa preta.

Entrada ou cenário	**Resultado esperado**
SARIF válido ou manifesto com a mesma localização	Identificação do mesmo endpoint e registro da origem da entrada.
Alerta sem localização válida ou fora do recorte	Diagnóstico explícito, com motivo da interrupção.
Resposta com diff inválido ou arquivo protegido	Proposta recusada antes da compilação e checkout original intacto.
Proposta que preserva a operação e passa nos testes de autorização	Resultados de segurança e funcionalidade aprovados. Aderência registrada em separado.
Proposta que bloqueia um usuário autorizado ou mantém acesso indevido	Reprovação registrada no critério correspondente.
Contexto insuficiente ou falha no ambiente	Incerteza, abstenção ou erro de infraestrutura identificado no relatório, conforme o caso.

Fonte elaborada pelos autores.

4.2 TESTES DE CAIXA BRANCA

A normalização será testada com regras mapeadas, regiões ambíguas e caminhos externos. A análise com Roslyn será testada com controladores, grupos de Minimal APIs, políticas padrão e de *fallback*, metadados `AllowAnonymous` e verificações por recurso. Os testes verificarão a resolução dos símbolos e a sinalização de relações que permanecem indeterminadas.

O recuperador será testado com endpoints de operações semelhantes sujeitos a regras diferentes. Os testes verificarão a ordenação, as referências de origem e o limite de contexto. Os testes de geração verificarão o conteúdo enviado nas condições A, B e C, a ausência dos arquivos protegidos na entrada e o tratamento de respostas

inválidas. O validador será testado com alterações fora do escopo, falha de compilação, regressões funcionais, falhas de autorização e interrupções durante a execução. Ao final de cada execução, os testes verificarão o descarte da cópia de trabalho e a integridade do checkout original.

4.3 CRITÉRIOS DE AVALIAÇÃO

a) **Segurança.** Os testes verificam a matriz esperada de acesso. Uma operação negada não deve produzir alteração de estado ou exposição de dados. Perfis e recursos de teste são definidos por caso.

b) **Funcionalidade.** O código deve compilar, as regressões funcionais devem passar e as identidades autorizadas devem conservar o comportamento esperado, incluindo os efeitos da operação.

c) **Aderência.** O *diff* deve reutilizar a política, o requisito ou o serviço aplicável ao caso, no nível apropriado, com evidência no repositório. A revisão examinará regras inventadas, verificações duplicadas, alterações fora do escopo e divergência em relação à regra de negócio documentada.

Os testes de integração do ASP.NET Core poderão usar `WebApplicationFactory` para iniciar a aplicação em um servidor de teste (Microsoft, s.d.a). Antes do experimento, o avaliador verificará que o conjunto de testes distingue o comportamento inicial do comportamento esperado e que a correção de referência preserva a operação autorizada. Os testes e a referência serão mantidos fora do contexto de geração. A revisão de aderência usará uma rubrica definida antes das execuções e omitirá a condição experimental do avaliador, quando possível.

4.4 DESENHO EXPERIMENTAL E MÉTRICAS

Cada caso elegível será executado nas três condições da Tabela 2, sempre a partir do mesmo commit. Modelo, versão, parâmetros, instruções comuns, limite de saída e procedimento de validação serão fixados. Será solicitada uma proposta por condição, sem devolver ao modelo resultados dos testes. O orçamento de recuperação

será calibrado no piloto e fixado antes de qualquer avaliação ampliada. Para B e C, o arquivo completo deve caber na entrada com a capacidade reservada para a recuperação em C. Casos que excederem esse limite serão registrados antes da comparação.

Tabela 2 – Condições de geração.

Condição	**Contexto de código enviado com o alerta**
A — Mínimo	Método ou manipulador do endpoint afetado.
B — Arquivo	Arquivo completo que contém o endpoint.
C — RAG	Mesmo arquivo completo de B e evidências de autorização recuperadas do repositório.

Fonte elaborada pelos autores.

A métrica principal será a fração de casos elegíveis em que a primeira proposta passa nos critérios de segurança, funcionalidade e aderência. A métrica de controle *pass@1* será a fração que passa nos critérios de segurança e funcionalidade, independentemente da aderência. Abstenções, propostas inválidas e falhas do patch contam como insucessos. Uma falha de infraestrutura poderá motivar nova execução com a mesma configuração. Se persistir e impedir a comparação, o caso será informado separadamente como incompleto, com motivo e contagem. O denominador das taxas será o número de casos com as três condições avaliáveis.

Na condição C, será registrada a precisão dos trechos recuperados, medida pela proporção de trechos relevantes entre os retornados, e a cobertura das evidências essenciais anotadas antes da geração. Ausência de contexto será registrada separadamente. O relatório também informará tempo de recuperação, geração e validação, uso de tokens e custo quando disponível. A análise apresentará contagens, taxas e diferenças pareadas entre condições, com destaque para C em relação a B. Os resultados do piloto servirão para verificar o desenho, incluindo a existência de casos que de fato dependam de evidências fora do arquivo. As conclusões serão limitadas aos casos e ao modelo avaliados.

5 ANÁLISE DE RISCOS

O recorte em autorização concentra o risco experimental na disponibilidade de casos reproduzíveis, na qualidade das evidências recuperadas e na confiabilidade da validação. Os riscos anteriores associados ao treinamento de classificadores e à geração de grafos deixam de compor o projeto. A Tabela 3 apresenta probabilidade (P), impacto (I) e grau de exposição. Os riscos serão revistos após o primeiro caso e após o piloto.

Tabela 3 – Riscos do Projeto Físico revisado.

Risco	P	I	Expos.	Redução	Contingência
Cobertura insuficiente do SAST	Alta	Alto	Crítica	Medir a cobertura por classe no piloto.	Usar manifestos e apresentar as origens dos casos separadamente.
Regra ambígua ou evidência no próprio arquivo	Média	Alto	Alta	Anotar a regra esperada e a localização das evidências antes das execuções.	Excluir casos sem regra verificável e relatar separadamente os casos autocontidos.
Política recuperada inadequada à operação	Média	Alto	Alta	Priorizar relações semânticas e equivalência da operação.	Registrar evidência insuficiente e permitir abstenção.
Validação insuficiente	Média	Alto	Alta	Manter testes externos à geração e verificar a matriz de acesso e os efeitos de estado.	Recusar mudanças em arquivos protegidos e revisar a cobertura dos testes.
Atraso no piloto	Alta	Alto	Crítica	Executar primeiro um caso completo e medir esforço e custo.	Concluir a comparação do piloto e reduzir a expansão.
Instabilidade ou custo do modelo	Média	Médio	Moderada	Fixar modelo e parâmetros, registrar uso e medir custo por condição.	Reduzir a expansão mantendo as três condições e a configuração.
Dados sensíveis ou execução insegura	Baixa	Alto	Moderada	Revisar os arquivos enviados e isolar build e testes sem rede nem credenciais.	Interromper a execução, descartar a cópia isolada e registrar o incidente.
Correções públicas vistas no treino	Média	Médio	Moderada	Registrar a origem e manter casos de avaliação preparados pela equipe.	Apresentar resultados por origem e discutir a possibilidade de contaminação.

6 CONCLUSÃO

A autorização em APIs ASP.NET Core depende de regras que podem estar distribuídas entre endpoints, configuração e serviços. Este Projeto Físico descreve uma CLI em C# que analisa essas relações com Roslyn, recupera evidências do repositório e usa um LLM para sugerir um *patch*. A proposta será examinada em ambiente isolado, com resultados de segurança, funcionalidade e aderência registrados separadamente para revisão humana.

O recorte é viável para um primeiro protótipo porque utiliza um único ecossistema, uma recuperação estrutural e textual e um fluxo comum de geração e validação. O primeiro caso testará o fluxo de ponta a ponta. A comparação entre contexto mínimo, arquivo completo e arquivo com recuperação permitirá avaliar a hipótese no piloto. A cobertura dos analisadores, a disponibilidade de regras verificáveis e o custo da avaliação determinarão a viabilidade da expansão do conjunto de casos.

REFERÊNCIAS

GitHub. **About autofix for code scanning**. 2026. Disponível em: https://docs.github.com/en/code-security/concepts/code-scanning/autofix-for-code-scanning. Acesso em: 10 nov. 2026.

GitHub. **SARIF support for code scanning**. s.d. Disponível em: https://docs.github.com/en/code-security/reference/code-scanning/sarif-files/sarif-support. Acesso em: 10 nov. 2026.

LEWIS, P.; PEREZ, E.; PIKTUS, A.; PETRONI, F.; KARPUKHIN, V.; GOYAL, N.; KÜTTLER, H.; LEWIS, M.; YIH, W.-t.; ROCKTÄSCHEL, T.; RIEDEL, S.; KIELA, D. Retrieval-augmented generation for knowledge-intensive NLP tasks. In: **Advances in Neural Information Processing Systems**. Curran Associates, Inc., 2020. v. 33, p. 9459–9474. Disponível em: https://arxiv.org/abs/2005.11401. Acesso em: 10 nov. 2026.

Microsoft. **Integration tests in ASP.NET Core**. s.d. Disponível em: https://learn.microsoft.com/en-us/aspnet/core/testing/integration-testing?view=aspnetcore-10.0. Acesso em: 10 nov. 2026.

Microsoft. **.NET Compiler Platform SDK concepts and object model**. s.d. Disponível em: https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/compiler-api-model. Acesso em: 10 nov. 2026.

Microsoft. **Policy-based authorization in ASP.NET Core**. s.d. Disponível em: https://learn.microsoft.com/en-us/aspnet/core/security/authorization/policies?view=aspnetcore-10.0. Acesso em: 10 nov. 2026.

Microsoft. **Resource-based authorization in ASP.NET Core**. s.d. Disponível em: https://learn.microsoft.com/en-us/aspnet/core/security/authorization/resource-based?view=aspnetcore-10.0. Acesso em: 10 nov. 2026.

MITRE. **CWE-285: Improper Authorization**. s.d. Disponível em: https://cwe.mitre.org/data/definitions/285.html. Acesso em: 10 nov. 2026.

MITRE. **CWE-862: Missing Authorization**. s.d. Disponível em: https://cwe.mitre.org/data/definitions/862.html. Acesso em: 10 nov. 2026.

MITRE. **CWE-863: Incorrect Authorization**. s.d. Disponível em: https://cwe.mitre.org/data/definitions/863.html. Acesso em: 10 nov. 2026.

YU, Z.; GUO, Z.; WU, Y.; YU, J.; XU, M.; MU, D.; CHEN, Y.; XING, X. PatchAgent: A practical program repair agent mimicking human expertise. In: **Proceedings of the 34th USENIX Security Symposium**. Seattle: USENIX Association, 2025. p. 4381–4400. Disponível em: https://www.usenix.org/conference/usenixsecurity25/presentation/yu-zheng. Acesso em: 10 nov. 2026.